

Session 3 · Deck 3.A4

Worked Example: The RBC Model, Fully Discretized

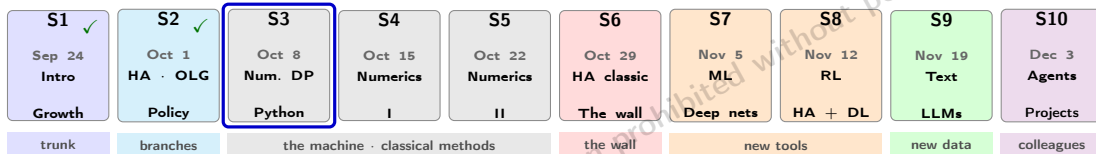
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.A4 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 3.A1 what it means to solve a model · 3.A2 value function iteration on a grid · 3.A3 Euler equations and time iteration · 3.A4 the worked example, end to end.

Hands-on, 3.B5:

1 Version 0: grid VFI against the closed form

2 Data → parameters: $g, n, \alpha, \delta, \beta$

3 The calibrated economy, solved and checked

4 Test the model: untar-getted ratio, Euler residual

The Model, and the One Thing We Know About It Exactly

The standard deterministic RBC problem, now with **realistic depreciation**:

$$\max \sum_{t=0}^{\infty} \beta^t \ln c_t \quad \text{s.t.} \quad c_t + k_{t+1} = Ak_t^\alpha + (1 - \delta)k_t.$$

Its steady state follows from $1 = \beta(\alpha Ak^{\alpha-1} + 1 - \delta)$:

$$k^* = \left(\frac{\alpha \beta A}{1 - \beta(1 - \delta)} \right)^{\frac{1}{1-\alpha}}.$$

At $A = 1$, $\alpha = 0.36$, $\beta = 0.95$, $\delta = 0.10$: $k^* = \mathbf{3.8219}$,
 $c^* = 1.2382$.

The $\delta = 1$ special case of 3.A1–3.A3 is the one with a closed-form policy.

What changed from this morning

$\delta = 0.10$, not 1. So there is a closed-form **steady state** — but *no* closed-form **policy**.

That is the ordinary situation, and it is why this example is worth doing. You will almost never have $g(k)$ to check against. You will very often have **one point** on it, and one point is enough to catch most mistakes.

“Fully Discretized” — What That Commits Us To

One decision does all the work: **the choice set is the state grid.**

$k' \in \{k_1, \dots, k_N\}$ = the same grid k lives on.

- No **interpolation**: v is only ever evaluated *at* a grid point, because k' is always a grid point.
- No **maximization routine**: the max is a search over N numbers, so argmax does it.
- No **root-finder**, no derivatives, no Jacobian.

Why start here

It is the only version of the algorithm with **no numerical method inside it**. Everything is indexing and arithmetic — so if the answer is wrong, the economics or the code is wrong, and nothing else can be blamed.

The price is the staircase of 3.A2, and an $O(N^2)$ inner loop. Next week buys both back.

Step 1: Parameters, the Benchmark, and the Grid

```
import numpy as np

A, alpha, beta, delta = 1.0, 0.36, 0.95, 0.10

# the one thing we know exactly
kss = (alpha*beta*A /
       (1 - beta*(1-delta)))*(1/(1-alpha))
# 3.8218909152179084

# the grid IS the choice set
N = 500
kg = np.linspace(0.25*kss, 1.75*kss, N)
h = kg[1] - kg[0] # 0.01149
```

- The benchmark is computed **first**, before any solver exists. If you write it afterwards you will unconsciously write it to agree.
- The grid is centred on k^* and runs $\pm 75\%$ — wide enough that the policy never touches an edge.
- $N = 500$ is a choice, not a fact. We will make it earn its keep.

Step 2: The Return Matrix, Built Once

```
# cash on hand at each state: (N,)
y = A*kg**alpha + (1-delta)*kg

# consumption for every (state, choice): (N,N)
C = y[:, None] - kg[None, :]

# u(c), with -1e10 where c <= 0
U = np.full_like(C, -1e10)
np.log(C, out=U, where=C > 0)
U[i, j] = u(Ak_i^alpha + (1 - delta)k_i - k_j).
```

Two ideas in six lines

Broadcasting builds every state against every choice at once — 3.B2's whole subject.

U **does not contain** v , so it is built once and reused every sweep.

The feasibility penalty must be a *number*, not $-\infty$: $-\infty + \beta v$ is $-\infty$, but a later difference such as $v_{n+1} - v_n$ is then $-\infty - (-\infty)$, which is `nan`, and `nan` loses every comparison silently.

Step 3: One Bellman Step Is Three Lines

```
M = U + beta*V[None, :] # (N,N)
Vn = M.max(axis=1)       # best value
pol = M.argmax(axis=1)   # best index
```

Read the shapes and the equation is there:

$$\underbrace{M_{ij}}_{(N,N)} = \underbrace{U_{ij}}_{\text{payoff}} + \beta \underbrace{v_j}_{\text{continuation}}, \quad v_i^{\text{new}} = \max_j M_{ij}.$$

The maximisation is a `max` along an axis. That is the entire numerical content of this method.

The axis is the whole game

`axis=1` reduces over *choices* and leaves one value per state. `axis=0` would reduce over states and return something with the right shape and no meaning.

Broadcasting will not warn you. **Assert the shape.**

`argmax` returns an *index*. Keeping the index rather than the value is what lets us read the policy off at the end.

Step 4: Iterate to the Fixed Point

```
V, it = np.zeros(N), 0
while True:
    M = U + beta*V[None, :]
    Vn = M.max(axis=1)
    pol = M.argmax(axis=1)
    d = np.max(np.abs(Vn - V))
    V, it = Vn, it + 1
    if d < 1e-6:
        break

print(it)                # 228
```

sweeps	228
$\ln \varepsilon / \ln \beta$	269
wall time	0.06 s
comparisons	57 million

228, not 269. The rule of thumb $\ln \varepsilon / \ln \beta$ is not a guarantee: the bound using the first step, $\|v_1 - v_0\| = 1.95$, says 284. It finishes early because the first steps shrink much faster than β (the second is $0.42\times$ the first); only the tail contracts at exactly 0.95.

Step 5: Read the Policy Off the Indices

```
g = kg[pol]          #  $k'(k)$ , on the grid  
c = y - g           #  $c(k)$ , by the budget
```

```
# the check we can actually run  
i = np.argmin(np.abs(g - kg))  
print(kg[i], kss)  
# 3.816146590195036 3.8218909152179084
```

`kg[pol]` is **fancy indexing**: `pol` is an array of indices, so this looks up all N of them at once.

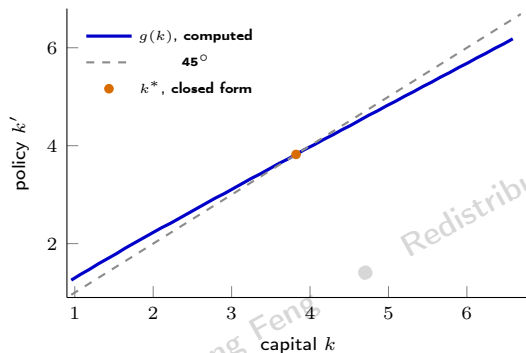
Consumption comes from the budget

Never solve for c separately. Recovering it from $c = y - g$ makes the budget constraint hold **exactly**, by construction, rather than to within a tolerance.

This is the whole solver: **five steps, about twenty-five lines**, no numerical library beyond NumPy, and not one routine we have not explained.

Does It Work? The 45-Degree Test

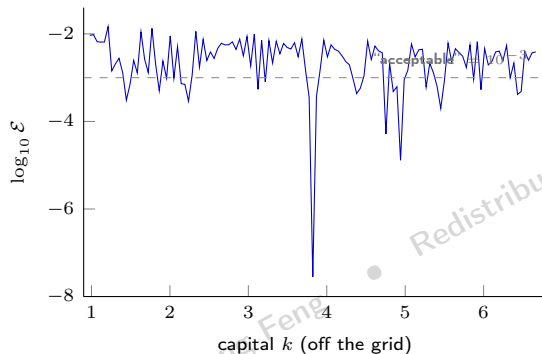
A steady state is a fixed point of the policy:
 $g(k^*) = k^*$. So the policy must cross the
45-degree line, and it must cross it **at the
number we computed in Step 1.**



k^* , closed form	3.8219
crossing, computed	3.8161
difference	-0.0057
grid spacing h	0.0115

The crossing is the **nearest grid point to k^*** — on a grid it can do no better. The solver is as accurate as the grid permits and no more, which is exactly the right answer.

The Honest Verdict: Euler Residuals



The 45-degree test checked *one point*. The residual checks **every** point, and it is less flattering:

log ₁₀ of mean $\mathcal{E}$	-2.48
worst log ₁₀ $\mathcal{E}$	-1.81

A worst case of $10^{-1.8}$ is a **1.6% consumption mistake** — sixteen times 3.A1's “acceptable” 0.1%.

And that is with $N = 500$ and a converged value function. The staircase does not care how long you iterate.

Version 1: Add a Shock, Make Three Changes

```
z = np.array([0.95, 1.05])
Pi = np.array([[0.9, 0.1],
               [0.1, 0.9]])

# (1) cash on hand gains an axis
y = z[None,:]*(A*kg**alpha)[: ,None] \
    + (1-delta)*kg[:,None]           # (N,2)
C = y[:, :,None] - kg[None, None, :] # (N,2,N)
U = np.full_like(C, -1e10)           # rebuilt
np.log(C, out=U, where=C > 0)         # as before

# (2) the expectation is a matrix product
EV = V @ Pi.T                       # (N,2)

# (3) one more axis to maximise over
M = U + beta*EV.T[None, :, :]
Vn, pol = M.max(axis=2), M.argmax(axis=2)
```

The iteration count is unchanged — it belongs to β , not to the state space (3.A2).

sweeps	228
wall time	0.13 s
return array	4.0 MB
$g(k^*)$, low z	3.7644
$g(k^*)$, high z	3.8793

The two policies **bracket** the deterministic $k^* = 3.8219$: a good shock saves more, a bad one saves less. A sign check you can do in your head, and it catches a reversed z grid immediately.

What We Deliberately Did Not Do

In this code	What it costs, measured here	
k' restricted to the grid	the crossing can only land within h of k^* ; worst residual $10^{-1.8}$	S4
No interpolation of v	no way to evaluate a candidate between grid points	S4
argmax over N candidates	57 million comparisons for one deterministic solve	S4
Every sweep a full sweep	228 of them, when the policy settles far earlier	S4
Π written by hand	no connection to any estimated AR(1)	S4
One continuous state	$(N, 2, N)$ is 4 MB at $N = 500$. A second endogenous state adds an axis today <i>and</i> a choice axis: $(N, N, 2, N, N)$, $N^2 = 250,000$ times larger — 1 TB	S5

Not one of these is a bug. They are the **cheapest possible choice** at each of the six decisions of 3.A1 — and this deck is the measurement that says what each cheap choice cost. Next week we buy them back one at a time and re-run exactly this example.

Takeaways

- The whole solver is **five steps and about twenty-five lines**: parameters and grid, return matrix, Bellman step, loop, read the policy. Nothing in it is a numerical method.
- **Compute the benchmark before the solver**. Here it is one number — $k^* = 3.8219$ — and one number is enough to catch most mistakes.
- The policy crossed the 45-degree line at **3.8161**, the nearest grid point to k^* . Right to within the grid, and no better: exactly what a correct implementation should give.
- One point passing is not the same as every point passing. The Euler residual over the whole grid is $10^{-1.8}$ at worst — **a 1.6% consumption mistake**, with a fully converged value function.
- Adding a shock changed **three lines** and the iteration count not at all. State space is memory; β is time.

Bridge: Now the Language It Is Written In

Everything in this deck was NumPy: an array, a broadcast, a reduction along an axis, and one piece of fancy indexing. Five constructs, and they carried the entire algorithm.

That is not a coincidence, and it is not something you can pick up by reading. Block B is those constructs from the ground up — plus the environment they run in, and the one library that will hand you derivatives when the next method needs them.

After the break (Block B)

3.B1 Your toolkit: Python, environments, version control.

3.B2 From loops to arrays — this deck's $\mathcal{U}$ and $\mathcal{M}$, explained properly.

3.B3 PyTorch, and coding with AI.

3.B4 The data workflow.